

Материалы кафедры ММП факультета ВМК МГУ. Практикум на ЭВМ

Занятие 01. Введение в Python, встроенные типы данных, модель памяти

Занятие провел: Ищенко Роман
(ischenkorv@my.msu.ru)

Материалы составил: Ищенко Роман
(ischenkorv@my.msu.ru)

Москва, Осенний семестр 2026

Источники:

- [python](#)
- [pyenv](#)
- [pep8 code style](#)

Управление версиями Python с помощью Pyenv

Pyenv – кроссплатформенный инструмент для управления версиями Python.

[GitHub](#)

Основные возможности:

- Установка/удаление версий Python. Версии существуют независимо и изолированно друг от друга. Это, в том числе, означает, что пакеты, установленные для одной версии, не будут видны для другой

- Позволяет установить глобальную версию Python для пользователя. Эта версия будет использована по умолчанию при выполнении команды **Python**
- Позволяет для каждого проекта указать специфическую версию Python. В каталоге проекта будет автоматически использована именно указанная версия

Достоинства:

- Не зависит от Python (то есть, для установки Pyenv в системе не требуется никакая «начальная» установленная версия Python)
- Требуется минимальной интеграцией со средой окружения (фактически, только добавления в переменную окружения PATH)
- Не требует прав администратора, может работать локально для отдельного пользователя

Настройка:

```
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.bashrc
echo '[[ -d $PYENV_ROOT/bin ]] && export
PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.bashrc
echo 'eval "$(pyenv init - bash)"' >> ~/.bashrc
```

Установка полностью локализована и изолирована. Удаление:

```
rm -rf ~/.pyenv
```

Для Windows отдельный проект: <https://github.com/pyenv-win/pyenv-win>

Использование:

```
pyenv <command> [<args>]
```

Несколько полезных команд:

- `versions` — список установленных версий
- `install -l` — список доступных для установки версий (там есть `anaconda`, `miniconda`)
- `install <version>` — установка версии в систему
- `global <version>` — выбор глобальной версии Python для пользователя
- `local <version>` — выбор версии Python для проекта (каталога)

Классическая первая программа

```
In [1]: print('Hello world!')
```

Hello world!

- не требуется специфических объявлений точки входа программы (`main` в C++) — код запускается с первой строки файла/ячейки

- `print` – стандартная функция языка, печатающая аргументы в стандартный поток вывода

Операторы присваивания

```
In [2]: a = 5  
print(a)
```

5

```
In [3]: a, b, c = 5, 6, 7  
print(a, b, c)
```

5 6 7

Ввод с клавиатуры

```
In [4]: f_name = input('Enter your first name:')  
l_name = input('Enter your last name:')  
print(f_name, l_name)
```

Enter your first name:317
Enter your last name:mmp
317 mmp

Выполнение кода на Python в виде строки

```
In [5]: eval('123 + 45')
```

Out[5]: 168

```
In [6]: exec(  
'''  
a = 12  
b = 3  
print(a ** b)  
'''  
print(a, b)
```

1728
12 3

- `eval` позволяет вычислить выражение и возвращает его результат (чистая функция)
- `exec` позволяет запустить любой фрагмент кода на Python (пример: ячейки Jupyter Notebook)

Объекты и переменные

```
In [7]: 4
```

```
Out[7]: 4
```

- создан объект типа `int`, который содержит значение 4
- поскольку на этот объект нет ссылок, он уничтожается после создания и печати

```
In [8]: a = 5
```

- создан объект типа `int`, в котором содержится значение 5
- затем создана переменная с именем `a`, которая ссылается на этот объект

```
In [9]: del a
```

- переменная `a` принудительно удалена
- объект, на который она ссылалась, тоже был удалён, поскольку больше на него нет ссылок

Объекты и переменные

```
In [10]: a = 5  
b = a
```

- создан объект типа `int`, в котором содержится значение 5
- затем создана переменная с именем `a`, которая ссылается на этот объект
- создана переменная `b`, которая указывает на тот же объект типа `int`

```
In [11]: a = a + 1 # equals to a += 1  
print(b)
```

5

- выражение `a = a + 1` создаёт новый объект типа `int`, равный 6
- переменная `a` начинает ссылаться на него и больше не связана с объектом, равным 5
- `b` продолжает ссылаться на него

Объекты и переменные в Python

- Переменные $\neq$ объекты

- Объект — это сущность, созданная и используемая в коде
- Объектами являются числа, строки, контейнеры, классы, функции и т.п.
- Переменная — это именованная ссылка на объект
- У одного объекта может быть много ссылок-переменных
- Объекты являются строго типизированными, ссылки — нет

Основные встроенные типы данных

- *логический, числовые, None*
- *строковые*
- *контейнерные: списки, словари, кортежи, множества*
- *файловый*
- *прочие: функции, классы, модули, сами типы*

Python:

- динамически типизируемый (сам следит за выводом типов)
- язык со строгой типизацией (объект заданного типа допускает только связанные с ним операции)

Тип объекта можно получить с помощью функции `type`

Логический тип

```
In [12]: True or False
```

```
Out[12]: True
```

```
In [13]: True or destroy_the_world()
```

```
Out[13]: True
```

```
In [14]: True and (False or True)
```

```
Out[14]: True
```

```
In [15]: bool()
```

```
Out[15]: False
```

```
In [16]: type(True)
```

```
Out[16]: bool
```

Целые числа

В Python тип `int` поддерживает бесконечно большие числа

```
In [17]: 2 + 5
```

```
Out[17]: 7
```

```
In [18]: 4 // 3
```

```
Out[18]: 1
```

```
In [19]: 4 % 3
```

```
Out[19]: 1
```

```
In [20]: 2 ** 100
```

```
Out[20]: 1267650600228229401496703205376
```

```
In [21]: type(1)
```

```
Out[21]: int
```

Литераты типа int

```
In [22]: 100, -20, 0 # decimal
```

```
Out[22]: (100, -20, 0)
```

```
In [23]: 0b11, 0B10 # binary
```

```
Out[23]: (3, 2)
```

```
In [24]: 0o11, 0011 # octal
```

```
Out[24]: (9, 9)
```

```
In [25]: 0x90A, 0X9F # hexadecimal
```

```
Out[25]: (2314, 159)
```

```
In [26]: bin(1000), oct(1000), hex(1000)
```

```
Out[26]: ('0b1111101000', '0o1750', '0x3e8')
```

Побитовые операции

Работаем с целыми числами как с битовыми массивами

```
In [27]: 1 << 2 # 001b -> 100b == 4d
```

```
Out[27]: 4
```

```
In [28]: 7 >> 1 # 111b -> 011b == 3d
```

```
Out[28]: 3
```

```
In [29]: 1 & 2 # 01b || 10b == 00b
```

```
Out[29]: 0
```

```
In [30]: 1 | 2 # 01b || 10b == 11b == 3d
```

```
Out[30]: 3
```

Числа с плавающей точкой

Вещественные числа в Python – это тип `double` из C

```
In [31]: 1.0 + 2
```

```
Out[31]: 3.0
```

```
In [32]: 4 / 3
```

```
Out[32]: 1.3333333333333333
```

```
In [33]: 3 ** 2.5
```

```
Out[33]: 15.588457268119896
```

```
In [34]: type(1.0)
```

```
Out[34]: float
```

```
In [35]: type(float())
```

```
Out[35]: float
```

```
In [36]: 0.1+0.1+0.1
```

```
Out[36]: 0.30000000000000004
```

Литераты числовых типов языка

Вещественные

```
In [37]: 1.3, 4., 1e+5, 1.0E+54
```

```
Out[37]: (1.3, 4.0, 100000.0, 1e+54)
```

Комплексные

```
In [38]: 3+4j, 2.0+1j, 5J
```

```
Out[38]: ((3+4j), (2+1j), 5j)
```

Расширение для дробных чисел

```
In [39]: import fractions  
a = fractions.Fraction(1, 3 ** 1000)  
len(str(a.denominator))
```

```
Out[39]: 478
```

Сравнение чисел

```
In [40]: x, y, z = 1, 2, 3
```

```
In [41]: x < y < z
```

```
Out[41]: True
```

```
In [42]: x < y >= z
```

```
Out[42]: False
```

```
In [43]: x < y != z
```

```
Out[43]: True
```

Округление вещественных чисел

```
In [44]: 11.0 // 3.0 # remove remainder
```

Out[44]: 3.0

```
In [45]: import math

print(math.trunc(4.7)) # move to zero
print(math.trunc(-4.7))
```

4
-4

```
In [46]: print(math.floor(4.7)) # move to lowest integer
print(math.floor(-4.7))
```

4
-5

```
In [47]: print(round(4.3)) # standart round
print(round(-4.7))
```

4
-5

Полезные встроенные функции для чисел

```
In [48]: pow(2, 4) == 2 ** 4
```

Out[48]: True

```
In [49]: abs(-2)
```

Out[49]: 2

```
In [50]: sum((1, 2, 3))
```

Out[50]: 6

```
In [51]: min(1, 2, -7, 44)
```

Out[51]: -7

```
In [52]: import math
print(math.sqrt(4))
print(math.sin(8))
```

2.0
0.9893582466233818

Тип None

```
In [53]: None == None
```

Out[53]: True

```
In [54]: None or True
```

```
Out[54]: True
```

```
In [55]: type(None)
```

```
Out[55]: NoneType
```

```
In [56]: 5 * None # strong typing
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[56], line 1  
----> 1 5 * None # strong typing  
  
TypeError: unsupported operand type(s) for *: 'int' and 'NoneType'
```

Строки

```
In [57]: s = 'qwerty'  
         type(s)
```

```
Out[57]: str
```

```
In [58]: len(s) # built-in function to get sequence length
```

```
Out[58]: 6
```

```
In [59]: s[0] # make a copy of element
```

```
Out[59]: 'q'
```

```
In [60]: s[-1] # make a copy of element
```

```
Out[60]: 'y'
```

```
In [61]: s[: -3: 1] # make a copy of elements from 0 to 3 excluding
```

```
Out[61]: 'qwe'
```

```
In [62]: s + '-add-on'
```

```
Out[62]: 'qwerty-add-on'
```

Подробнее об индексировании и срезах

```
In [63]: s = 'qwerty'  
         s[0]
```

```
Out[63]: 'q'
```

```
In [64]: s[0: 10]
```

```
Out[64]: 'qwerty'
```

```
In [65]: s[slice(1, 3)]
```

```
Out[65]: 'we'
```

```
In [66]: s[:10:2] # QwErTy____
```

```
Out[66]: 'qet'
```

```
In [67]: s[::-1]
```

```
Out[67]: 'ytrewq'
```

Операции над строками

```
In [68]: s + 5 # strong typing
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[68], line 1
----> 1 s + 5 # strong typing

TypeError: can only concatenate str (not "int") to str
```

```
In [69]: s * 5 # strong typing
```

```
Out[69]: 'qwertyqwertyqwertyqwertyqwerty'
```

```
In [70]: s[0] = 'a' # str type is immutable
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[70], line 1
----> 1 s[0] = 'a' # str type is immutable

TypeError: 'str' object does not support item assignment
```

Операции над строками

```
In [71]: s = 'Qwe{0}{1}{0}'.format('rty', 3)
s
```

```
Out[71]: 'Qwerty3rty'
```

```
In [72]: print(s.find('w'), s.find('9'))
```

```
1 -1
```

```
In [73]: s.split('r')
```

```
Out[73]: ['Qwe', 'ty3', 'ty']
```

```
In [74]: '-'.join(['New', 'York'])
```

```
Out[74]: 'New-York'
```

```
In [75]: s.replace('ty', 'TY')
```

```
Out[75]: 'QwerTY3rTY'
```

```
In [76]: s[0].isupper()
```

```
Out[76]: True
```

Операции над строками

Две новые простые операции были добавлены в Python 3.9:

```
In [77]: 'prefix_string'.removeprefix('prefix_')
```

```
Out[77]: 'string'
```

```
In [78]: 'string_suffix'.removesuffix('_suffix')
```

```
Out[78]: 'string'
```

Как ещё можно задавать строки

```
In [79]: 'qwe\'rty'
```

```
Out[79]: "qwe'rty"
```

```
In [80]: "qwe'rty"
```

```
Out[80]: "qwe'rty"
```

```
In [81]: '''qwe'rty'''
```

```
Out[81]: "qwe'rty"
```

```
In [82]: 'qwe\0rty' # print hex code in '\xNN' format for non-printable characters
```

```
Out[82]: 'qwe\x00rty'
```

```
In [83]: 'qwe\nrty'
```

```
Out[83]: 'qwe\nrty'
```

```
In [84]: print('qwe\nrty') # we'll discuss difference between __repr__ and __str__ t
```

qwe
rty

Форматирование строк

В Python есть похожих способа форматирования – выражение и метод

```
In [85]: 'x = %d, y = %f' % (10.5, 11)
```

```
Out[85]: 'x = 10, y = 11.000000'
```

```
In [86]: 'x = %o, y = %E' % (8, 1.0 / 3) # octal format, exponential + upper case
```

```
Out[86]: 'x = 10, y = 3.333333E-01'
```

```
In [87]: 'x = %10.2f' % (1.0 / 3) # min width and precision
```

```
Out[87]: 'x =          0.33'
```

```
In [88]: # add leading zeros if len < min width
s = ''
x = %(value_2)010.2f
x = %(value_1)010.2f
''' % ({'value_1': 10000, 'value_2': 0.12345})

print(s) # __str__
s        # __repr__
```

```
x = 0000000.12
```

```
x = 0010000.00
```

```
Out[88]: '\nx = 0000000.12\nx = 0010000.00\n'
```

Форматирование строк

```
In [89]: 'x = {}, y = {}'.format(10, 20)
```

```
Out[89]: 'x = 10, y = 20'
```

```
In [90]: 'x = {1}, y = {0}'.format(10, 20)
```

```
Out[90]: 'x = 20, y = 10'
```

```
In [91]: f'x = {10}, y = {20}'
```

```
Out[91]: 'x = 10, y = 20'
```

```
In [92]: 'x = {val_1:f}, y = {val_2:010.2f}'.format(val_1=10, val_2=10)
```

```
Out[92]: 'x = 10.000000, y = 0000010.00'
```

```
In [93]: import sys
         'platform: {sys_var.platform}'.format(sys_var=sys)
```

```
Out[93]: 'platform: linux'
```

Оператор присваивания в форматных строках

Синтаксический сахар, добавленный в версии Python 3.8:

```
In [94]: from datetime import date
         from math import cos, radians

         user = 'eric_idle'
         member_since = date(1975, 7, 31)

         f'{user=} {member_since=}'
```

```
Out[94]: "user='eric_idle' member_since=datetime.date(1975, 7, 31)"
```

Можно форматировать вывод:

```
In [95]: delta = date.today() - member_since
         f'{user=!s} {delta.days=:d}'
```

```
Out[95]: 'user=eric_idle delta.days=18683'
```

Можно вставлять вычисления и выводить как результат, так и вычисляющий код:

```
In [96]: theta = 4
         f'{theta=} {cos(radians(theta))=:.3f}'
```

```
Out[96]: 'theta=4 cos(radians(theta))=0.998'
```

Строка как набор байтов

```
In [97]: b = bytearray(b'qwerty')
         type(b)
```

```
Out[97]: bytearray
```

```
In [98]: b[1] = ord('a')
         b
```

```
Out[98]: bytearray(b'qaerty')
```

- `bytearray` – массив байтов, в который можно записывать номера 8-битных символов
- в отличие от `str`, объекты этого типа являются изменяемыми
- функция `ord` возвращает по символу его номер в таблице ASCII

Контейнерные типы: списки

```
In [99]: lst = [1, None, 'qwerty']  
list([1, None, 'qwerty']) == lst # list() gets any iterable object as input
```

Out[99]: True

```
In [100]: lst[0] = [10, 20]  
lst
```

Out[100]: [[10, 20], None, 'qwerty']

- список – более общий аналог массива (не путать со структурой данных "список")
- элементы могут быть произвольного типа (в т.ч. и ссылками)
- сам список является изменяемым типом

Списки

```
In [101]: len(lst)
```

Out[101]: 3

```
In [102]: type(lst)
```

Out[102]: list

```
In [103]: lst[-1]
```

Out[103]: 'qwerty'

```
In [104]: lst[: 100] # slice bounds can be exceed real ones
```

Out[104]: [[10, 20], None, 'qwerty']

```
In [105]: lst[5]
```

```
-----  
IndexError                                Traceback (most recent call last)  
Cell In[105], line 1  
----> 1 lst[5]  
  
IndexError: list index out of range
```

Операции над списками

```
In [106]: lst = [1, 2, 3]  
lst.append(4)
```

```
lst
```

```
Out[106]: [1, 2, 3, 4]
```

```
In [107... print(lst.pop(), lst)
```

```
4 [1, 2, 3]
```

```
In [108... lst.sort(reverse=True) # equals to lst.reverse()  
lst
```

```
Out[108]: [3, 2, 1]
```

```
In [109... lst * 2
```

```
Out[109]: [3, 2, 1, 3, 2, 1]
```

```
In [110... lst + list('abc')
```

```
Out[110]: [3, 2, 1, 'a', 'b', 'c']
```

Списковые включения и генераторы списков

```
In [111... [x ** 2 for x in [1, 2, 3]] # list comprehension
```

```
Out[111]: [1, 4, 9]
```

```
In [112... lst = [x ** 2 for x in [1, 2, 3] if x > 1]  
print(type(lst))  
  
for _ in [1, 2]: # _ means value we really don't need  
    for e in lst:  
        print(e)
```

```
<class 'list'>
```

```
4
```

```
9
```

```
4
```

```
9
```

```
In [113... lst = (x ** 2 for x in [1, 2, 3] if x > 1) # list generator  
print(type(lst))  
print(lst)  
  
for _ in [1, 2]:  
    for e in lst:  
        print(e)  
    print('-')
```

```
<class 'generator'>
<generator object <genexpr> at 0x7fe3dc2885f0>
4
9
-
-
```

Снова про объекты и переменные

```
In [114... a = [2]
b = a
a += [1]
# a = a + [1]
print(b)
```

[2, 1]

- список – изменяемый объект, создание новой ссылки не приводит к появлению нового объекта
- есть две переменные, указывающие на один и тот же список

```
In [115... del a
print(b)
```

[2, 1]

- удаление `a` не приводит к удалению списка, поскольку на него ещё ссылается `b`

Копирование изменяемых объектов

```
In [116... a = [1, 2, 3]
b = a[:]
b.append(4)
print(a)
```

[1, 2, 3]

```
In [117... a = [1, 2, 3]
b = a.copy()
b.append(4)
print(a)
```

[1, 2, 3]

Копирование вложенных изменяемых объектов

```
In [118... a = [['a', 'b', 'c'], 2, 3]
b = a.copy()
b[0].append('d')
print(a)
print(b)
```

```
[['a', 'b', 'c', 'd'], 2, 3]
[['a', 'b', 'c', 'd'], 2, 3]
```

```
In [119... import copy

a = [['a', 'b', 'c'], 2, 3]
b = copy.deepcopy(a)
b[0].append('d')
print(a)
print(b)
```

```
[['a', 'b', 'c'], 2, 3]
[['a', 'b', 'c', 'd'], 2, 3]
```

Контейнерные типы: словари

- словарь (`dict`) – это ассоциативный массив (отображение), т.е. набор пар "ключ-значение"
- словарь поддерживает возможность быстро искать значение по ключу
- в Python словари реализованы на основе хэш-таблиц (сложность поиска $O(1)$)
- словари также являются изменяемым типом данных
- значения могут иметь произвольный тип
- ключи – произвольный неизменяемый тип

Словари

```
In [120... type(dict()), type({})
```

```
Out[120]: (dict, dict)
```

```
In [121... # simple creation of nested structures!
d = {'1': 1, 's': 2, None: [1, {None: True}]}
d
```

```
Out[121]: {'1': 1, 's': 2, None: [1, {None: True}]}
```

```
In [122... d['s'] = 'two'
d
```

```
Out[122]: {'1': 1, 's': 'two', None: [1, {None: True}]}
```

```
In [123... del d['1'] # remove object
print(d)
```

```
{'s': 'two', None: [1, {None: True}]}
```

```
In [124... d2 = {('s', 3), ('a', 'b')}
```

Операции над словарями

```
In [125... d = dict.fromkeys(list('abcccc'))  
d
```

```
Out[125]: {'a': None, 'b': None, 'c': None}
```

```
In [126... for k in d.keys(): # view object  
    d[k] = ord(k)  
d
```

```
Out[126]: {'a': 97, 'b': 98, 'c': 99}
```

```
In [127... for k, v in d.items(): # view object  
    print(k, v)
```

```
a 97  
b 98  
c 99
```

```
In [128... D = dict(a=ord('a'), b=ord('b'), c=ord('c'))  
d == D
```

```
Out[128]: True
```

```
In [129... d2 = {'d': 13}  
d.update(d2)  
d
```

```
Out[129]: {'a': 97, 'b': 98, 'c': 99, 'd': 13}
```

Операции над словарями

Новый оператор слияния словарей был введен начиная с Python 3.9:

```
In [130... d_1, d_2 = {1: 'a', 2: 'b'}, {3: 'c', 4: 'd'}  
d_1 | d_2
```

```
Out[130]: {1: 'a', 2: 'b', 3: 'c', 4: 'd'}
```

```
In [131... d_2 = {1: 'c'}  
d_1 | d_2
```

```
Out[131]: {1: 'c', 2: 'b'}
```

```
In [133... d_1 | [(3, 'c'), (4, 'd')] # -> TypeError: unsupported operand type(s) for |
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[133], line 1  
----> 1 d_1 | [(3, 'c'), (4, 'd')] # -> TypeError: unsupported operand type  
(s) for |: 'dict' and 'list'  
  
TypeError: unsupported operand type(s) for |: 'dict' and 'list'
```

```
In [134... d_1, d_2 = {1: 'a', 2: 'b'}, {3: 'c', 4: 'd'}  
          d_1 |= d_2  
          d_1
```

```
Out[134]: {1: 'a', 2: 'b', 3: 'c', 4: 'd'}
```

```
In [135... d_1 = {1: 'a', 2: 'b'}  
          d_1 |= [(3, 'c'), (4, 'd')]  
          d_1
```

```
Out[135]: {1: 'a', 2: 'b', 3: 'c', 4: 'd'}
```

Контейнерные типы: кортежи

- очень похожи на списки, но являются неизменяемыми

```
In [136... type(), type(tuple())
```

```
Out[136]: (tuple, tuple)
```

```
In [137... (1, 2, 'str', 4.0, None)
```

```
Out[137]: (1, 2, 'str', 4.0, None)
```

```
In [138... (1, 2) + (3.0, 6, None)
```

```
Out[138]: (1, 2, 3.0, 6, None)
```

```
In [139... tpl = tuple([1, 2, 3])  
          tpl[2]
```

```
Out[139]: 3
```

Контейнерные типы: множества

- Как и словари, основаны на хэш-таблицах
- Элементы должны быть неизменяемыми

```
In [140... type(set()), type({1}), type({}) # Empty '{}' is a dict, not a set!
```

```
Out[140]: (set, set, dict)
```

```
In [141... {1, 2, 3, 3, 2, 1, 'set', 'a'}
```

```
Out[141]: {1, 2, 3, 'a', 'set'}
```

```
In [142... s = {1, 2, 3}
s.add(4)
s
```

```
Out[142]: {1, 2, 3, 4}
```

```
In [143... s1, s2 = {1, 2, 3}, set([3, 4, 5])

print('{}\t{}\t{}\t{}'.format(s1 - s2, s1 | s2, s1 & s2, s1 < s2))

{1, 2} {1, 2, 3, 4, 5} {3} False
```

Операции над множествами

```
In [144... A, B = set('abc'), {'a', 'c', 'd'}
```

```
In [145... A - B # minus: in A and not in B (== A.difference(B))
```

```
Out[145]: {'b'}
```

```
In [146... A | B # union: in A or in B (== A.union(B))
```

```
Out[146]: {'a', 'b', 'c', 'd'}
```

```
In [147... A & B # intersection: in A and in B (== A.intersection(B))
```

```
Out[147]: {'a', 'c'}
```

```
In [148... A ^ B # sym diff: (in A and not in B) and via versa (== A.symmetric_difference(B))
```

```
Out[148]: {'b', 'd'}
```

Операции над множествами

```
In [149... A.add('e')
A
```

```
Out[149]: {'a', 'b', 'c', 'e'}
```

```
In [150... A.update(B)
A
```

```
Out[150]: {'a', 'b', 'c', 'd', 'e'}
```

```
In [151... A.remove('a')  
A
```

```
Out[151]: {'b', 'c', 'd', 'e'}
```

```
In [152... A.discard('a')  
A
```

```
Out[152]: {'b', 'c', 'd', 'e'}
```

```
In [153... 'a' in A
```

```
Out[153]: False
```

```
In [154... {'a', 'b'} < A # is subset (== {'a', 'b'}.issubset(A))
```

```
Out[154]: False
```

Файловый тип

```
In [164... !echo "line1" > some_file.txt  
!echo "line317" >> some_file.txt
```

```
In [165... fin = open('some_file.txt', 'r') # 'r' can be avoided  
print(fin.read())  
fin.close()  
fin.closed
```

```
line1  
line317
```

```
Out[165]: True
```

```
In [166... fin = open('some_file.txt', 'r') # 'r' can be avoided  
  
# strip removes blank and special characters from edges  
lines = [line.strip() for line in fin.readlines()]  
  
print(' '.join(lines))  
fin.close()  
fin.closed
```

```
line1 line317
```

```
Out[166]: True
```

Файловый менеджер контекста

```
In [167... fin = open('some_file.txt', 'r') # 'r' can be avoided  
print(fin.read(), 0/0)  
fin.close()
```

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[167], line 2
      1 fin = open('some_file.txt', 'r') # 'r' can be avoided
----> 2 print(fin.read(), 0/0)
      3 fin.close()

ZeroDivisionError: division by zero
```

In [168... `fin.closed`

Out[168]: False

In [169... `with open('some_file.txt', 'r') as fin:`
`print(fin.read(), 0/0)`

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[169], line 2
      1 with open('some_file.txt', 'r') as fin:
----> 2     print(fin.read(), 0/0)

ZeroDivisionError: division by zero
```

In [170... `fin.closed`

Out[170]: True

Запись в файл

- отступы используются для обозначения операторных блоков (вместо фигурных скобок в C++)

In [171... `with open('some_file.txt', 'a') as fout:`
`fout.write('additional string')`

In [172... `with open('some_file.txt', 'r') as fin:`
`print(fin.read())`

```
line1
line317
additional string
```

In [173... `with open('some_file.txt', 'w') as fout:`
`fout.write('new\ncontent')`

In [174... `with open('some_file.txt', 'r') as fin:`
`print(fin.read())`

```
new
content
```

Содержимое файлов как массив байтов

Полезно при работе с сериализаторами

```
In [175... s = bytearray('строка', 'utf-8')
with open('tmp.bin', 'wb') as fout:
    fout.write(s)
```

```
In [176... with open('tmp.bin', 'r') as fin:
    print(list(fin.read()))
```

```
['c', 'т', 'р', 'о', 'к', 'а']
```

```
In [177... with open('tmp.bin', 'rb') as fin:
    print(list(fin.read()))
```

```
[209, 129, 209, 130, 209, 128, 208, 190, 208, 186, 208, 176]
```

```
In [178... len('з'.encode('utf-8'))
```

```
Out[178]: 2
```

Условный оператор if

```
In [179... if 3 % 3 == 0:
    print('success')
```

```
success
```

```
In [180... a = 5
if a == 2:
    print(2)
elif a == 4:
    print(4)
else:
    print(a)
```

```
5
```

```
In [181... print(a if a > 5 else 2.5) # ternary operator
```

```
2.5
```

Оператор цикла while

```
In [182... a = 5
while a > 4:
    print('Current a: {}'.format(a))
    a -= 1
```

```
Current a: 5
```

```
In [183... while True: # infinity loop
    pass # in Python 3 the equal statement is '...'
```

```
-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[183], line 2
      1 while True: # infinity loop
----> 2     pass # in Python 3 the equal statement is '...'

KeyboardInterrupt:
```

```
In [184... a = 6
while a > 2:
    a -= 1
    if a == 5: continue
    if a == 3: break
    print(a)
else:
    print('else')
```

4

- `pass` – оператор пустого действия
- `continue` – досрочный переход на следующую итерацию цикла
- `break` – досрочный выход из цикла
- `else` – обозначает секцию, которая выполняется, если цикл был завершён не по `break`

Оператор цикла for

- в отличие от C/C++, в Python чаще всего используются range-based for-циклы
- классические циклы по индексам так же доступны, но при прочих равных менее предпочтительны

```
In [185... lst = ['a', 'b', 'c']
for element in lst: # range-based for
    print(element)
type(range(5))
```

a
b
c

Out[185]: range

```
In [186... for index in range(len(lst)): # index-based for
    print(lst[index])
```

a
b
c

- функция `range` создаёт генератор, позволяющий итерироваться по

последовательности чисел с заданными границами и шагом

```
In [187... for x, y in [(1, 'a'), (2, 'b')]:
    print(x, y)
else:
    print('For-loop ended without break!')
```

```
1 a
2 b
For-loop ended without break!
```

```
In [188... for x, y, z in [(1, 2, 3), ('a', 'b', 'c')], [True, False, None]]:
    if isinstance(x, int):
        continue
    print(x, y, z)
```

```
a b c
```

- `isinstance` – функция языка, позволяющая проверить принадлежность объекта типу (учитывает иерархию наследования типов)

Оператор walrus

- "Моржовый" оператор был добавлен в Python начиная с версии 3.8
- Он является присваивающим *выражением*, т.е., в отличие от *оператора* присваивания, он возвращает присвоенное значение

```
In [189... if (n := len('test')) > 3:
    print(f'String is too long ({n}))')
```

```
String is too long (4)
```

Иногда он может уменьшить объем вычислений:

```
In [190... [x ** 2 for x in range(10) if x ** 2 % 2 == 0]
```

```
Out[190]: [0, 4, 16, 36, 64]
```

```
In [191... [y for x in range(10) if (y := x ** 2) % 2 == 0]
```

```
Out[191]: [0, 4, 16, 36, 64]
```

Конструкция match/case

Добавлена в Python 3.10

```
In [192... value = '1'
if value == '1':
    print('One')
```

```
elif value == '2':  
    print('Two')  
else:  
    print('Many')
```

One

```
In [ ]: value = '1'  
match value:  
    case '1':  
        print('One')  
    case '2':  
        print('Two')  
    case _:  
        print('Many')
```

Пока разница небольшая, чуть сокращённый синтаксис, но...

```
In [194... value = None  
if not value:  
    print('No')  
else:  
    print('Yes')
```

No

```
In [ ]: value = None  
match value:  
    case '':  
        print('No')  
    case _:  
        print('Yes')
```

Здесь уже видим разницу.

Что на самом деле происходит под капотом:

```
In [195... value = '1'  
if isinstance(value, str) and value == '1':  
    print('One')  
elif isinstance(value, str) and value == '2':  
    print('Two')  
else:  
    print('Many')
```

One

```
In [ ]: value = '1'  
match value:  
    case '1':  
        print('One')  
    case '2':  
        print('Two')  
    case _:  
        print('Many')
```

Но на этом функции не ограничиваются. Можно записывать несколько вариантов:

```
In [ ]: value = '1'
match value:
    case '1' | '2':
        print('Match')
```

Можно проверять на соответствие структурам:

```
In [ ]: value = 'One Two Three Four'.split()

match value:
    case a, *b, c:
        print(b, a)
    case _:
        print('Not match')
```

```
In [ ]: value = 'One Two Three'.split()
match value:
    case a, b:
        print(b, a)
    case a, b, c:
        print(c, b, a)
    case _:
        print('Not match')
```

```
In [ ]: value = 'One Two Three'.split()
match value:
    case 'One', *b:
        print(*b)
    case a, b, c:
        print(c, b, a)
    case _:
        print('Not match')
```

```
In [ ]: value = 'One 3 Three'.split()
match value:
    case 'One', '1' | '2', *b:
        print(*b)
    case a, b, c:
        print(c, b, a)
    case _:
        print('Not match')
```

Именовывать совпавшие части:

```
In [ ]: value = 'One 2 Three'.split()
match value:
    case 'One', '1' | '2' as num, *b:
        print(num, *b)
    case a, b, c:
        print(c, b, a)
```

```
case _:
    print('Not match')
```

```
In [ ]: value = 'One 3 Three Four'.split()
match value:
    case a, *b, c:
        print(a, b, c)
    case _:
        print('Not match')
```

In []: И даже сопоставлять словари:

```
In [ ]: value = {'a': 2, 'b': 3}
match value:
    case {'a': 1 | 2 as a, 'b': b}:
        print(a, b)
    case _:
        print('Other')
```

Атрибуты объекта

- любой объект в Python имеет атрибуты: набор полей и методов, определяющих свойства объекта и способы работы с ним
- атрибуты можно читать, устанавливать и менять
- список всех атрибутов получается с помощью встроенной функции `dir`

```
In [196... print(len(dir(5)))
print(dir(5))
```

```
71
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__delattr__', '__dir__', '__divmod__', '__doc__', '__eq__', '__float__', '__floor__', '__floordiv__', '__format__', '__ge__', '__getattr__', '__getnewargs__', '__gt__', '__hash__', '__index__', '__init__', '__init_subclass__', '__int__', '__invert__', '__le__', '__lshift__', '__lt__', '__mod__', '__mul__', '__ne__', '__neg__', '__new__', '__or__', '__pos__', '__pow__', '__radd__', '__rand__', '__rdivmod__', '__reduce__', '__reduce_ex__', '__repr__', '__rfloordiv__', '__rlshift__', '__rmod__', '__rmul__', '__ror__', '__round__', '__rpow__', '__rrshift__', '__rshift__', '__rsub__', '__rtruediv__', '__rxor__', '__setattr__', '__sizeof__', '__str__', '__sub__', '__subclasshook__', '__truediv__', '__trunc__', '__xor__', 'as_integer_ratio', 'bit_length', 'conjugate', 'denominator', 'from_bytes', 'imag', 'numerator', 'real', 'to_bytes']
```

```
In [197... a = 5
a.__add__(1) # exactly the same as 'a + 1'
```

Out[197]: 6

Как правильно называть переменные

- `regular_variable`
- `ClassName`
- `SOME_CONSTANT_10`
- `_private_class_field_or_method`
- `__avoid_overriding_of_this_field_or_method_in_childrens`
- `__magic_method__` or `__special_field__`

Не используйте переменные с unicode-символами:

- `плохое_имя_для_переменной`

Модули

- любой файл на Python представляет собой модуль
- импорт модуля вызовет выполнение содержащегося в нём кода

```
In [198... with open('test_module.py', 'w') as fout:
    fout.write('print(10)\n')
    fout.write('def new_print(a):\n\tprint(a)\n')
```

- создали текстовый файл и записали в него оператор печати и определение функции
- ключевое слово `def` используется для определения функций и классов (не единственный способ)

```
In [199... import test_module
import test_module
```

10

- импортируем модуль, операторы выполняются

В стандартной библиотеке языка есть много полезных модулей – надо пользоваться

Пространства имён модулей

```
In [200... new_print(5)
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[200], line 1  
----> 1 new_print(5)  
  
NameError: name 'new_print' is not defined
```

```
In [201... test_module.new_print(5)
```

5

```
In [202... from test_module import new_print  
new_print(5)
```

5

Повторная загрузка модуля

- если модуль был один раз загружен (исполнен), второй импорт ничего не произведёт
- это верно даже в том случае, если модуль был изменён после импорта

```
In [203... import test_module # no any output!
```

- но модуль можно перезагрузить принудительно

```
In [204... from importlib import reload  
reload(test_module)
```

10

```
Out[204]: <module 'test_module' from '/home/alexandr/Документы/Prepod_work/mmp_practicum_fall/Seminars/01-intro-python/test_module.py'>
```

Компиляция кода на Python

- Код на языке Python перед выполнением транслируется в байт-код
- Это байт-код выполняется виртуальной машиной языка, в ходе чего он переводится в машинные команды
- Генерация байт-кода при выполнении кода скрипта выполняется на лету
- Однако в случае импортирования кода из модуля байт-код этого модуля сохраняется в файлы с расширением `.pyc`
- В дальнейшем это позволяет ускорить загрузку модуля (но не скорость выполнения!)

- Необходимость регенерации файла `.рус` определяется по времени последнего изменения этого файла и его источника
- Хорошей практикой является игнорирование этих файлов и автоматическая регенерация при каждом запуске в новом окружении
- Помимо файлов `.рус` генерироваться могут файлы `.pyo` (запуск с оптимизацией) и `.pyd` (файл типа dll в ОС Windows)

Модуль collections

Стандартная задача: подсчёт числа элементов

```
In [205... s = 'GDJKFHGKJXBZJVJHBJZBXXXXXXXXXXXXXXXXXXXXHFG'
```

```
In [206... d = {}
for element in s:
    if not element in d:
        d[element] = 0
    d[element] += 1

result = ''
for k, v in d.items():
    result += f'({k}: {v}) '
print(result.strip())
```

```
(G: 3) (D: 1) (J: 4) (K: 2) (F: 3) (H: 3) (X: 20) (B: 3) (Z: 2) (V: 1)
```

```
In [207... from collections import Counter

for k, v in Counter(s).items():
    print(f'({k}: {v}) ', end='') # print can end not only with '\n'
print()
```

```
(G: 3) (D: 1) (J: 4) (K: 2) (F: 3) (H: 3) (X: 20) (B: 3) (Z: 2) (V: 1)
```

Модуль os

`os` содержит разнообразные функции и библиотеки для широкого круга задач:

- работа с переменными окружения
- работа с процессами
- работа с файловой системой
- работы с путями в файловой системе (библиотека `os.path`)

```
In [208... import os

len(os.listdir('.'))
```

Out[208]: 7

```
In [209... os.mkdir('temp')
os.rmdir('temp')

os.mkdir('temp')
os.mkdir('temp/temp')
#os.rmdir('temp') -> OSError: [Errno 39] Directory not empty: 'temp'
```

```
In [210... import shutil

shutil.rmtree('temp')
```

Библиотека os.path

```
In [211... print(os.path.exists('01-intro.ipynb'))
print(os.path.isfile('01-intro.ipynb'))
print(os.path.isdir('01-intro.ipynb'))
```

True
True
False

```
In [212... addr = os.path.dirname(os.path.abspath('01_intro.ipynb'))
print(addr)
```

/home/alexandr/Документы/Prepod_work/mmp_practicum_fall/Seminars/01-intro-python

```
In [213... os.path.join(*addr.split('/')) # f(*[a, b, c]) -> f(a, b, c)
```

Out[213]: 'home/alexandr/Документы/Prepod_work/mmp_practicum_fall/Seminars/01-intro-python'

pathlib

pathlib – более удобный модуль для работы с файловой системой

- До Python 3.4 работа с ФС производилась с помощью `os` и `glob`
- Теперь удобнее и нагляднее пользоваться модулем `pathlib`:

```
In [214... os.makedirs(os.path.join('temp', 'temp'), exist_ok=True)
os.rename('temp', 'new_temp')
shutil.rmtree('new_temp')
```

```
In [215... from pathlib import Path

Path('temp/temp').mkdir(parents=True, exist_ok=True)
Path('temp').rename('new_temp')
shutil.rmtree('new_temp')
```

- Вложенные вызовы тяжелее читать, чем последовательные
- Модуль `shutil` все ещё необходим для удаления непустых директорий, перемещения и копирования

pathlib: пример

Считывание текста из всех `.ipynb`-файлов в текущей директории с помощью `glob`:

```
In [222... from glob import glob

contents_1 = []
for filename in glob('*.ipynb', recursive=False):
    with open(filename) as fin:
        contents_1.append(fin.read())
```

```
In [223... len(contents_1), len(contents_1[0])
```

```
Out[223]: (1, 121961)
```

И с помощью `pathlib`:

```
In [224... from pathlib import Path

contents_2 = [path.read_text() for path in Path.cwd().glob('*.ipynb')]

contents_1 == contents_2
```

```
Out[224]: True
```

pathlib: итоговые замечания

- Использование `pathlib` улучшает качество кода и является рекомендованным стандартом для работы с ФС
- `pathlib.Path` самостоятельно производит нормализацию путей в зависимости от ОС
- Встроенные функции в Python 3.6 и выше, принимающие на вход строки-пути, умеют обрабатывать и объекты `Path`
- `pathlib` уступает в скорости `os` в 2-4 раза, поэтому не подходит для фрагментов кода с очень интенсивной работой с ФС
- В новых версиях Python обеспечивается легкая взаимозаменяемость путей-строк и объектов `Path`:

```
In [225... path_str = os.path.join('temp', 'temp')
path_path = Path('temp/temp')
```

```
In [226... from os import fspath

a, b = fspath(path_str), fspath(path_path)
print(a, b, type(a), type(b))

temp/temp temp/temp <class 'str'> <class 'str'>
```

```
In [227... Path(path_str), Path(path_path)
```

```
Out[227]: (PosixPath('temp/temp'), PosixPath('temp/temp'))
```

Модуль re для регулярных выражений

```
In [228... import re
```

- `[]` – множество допустимых символов
- `-` – обозначает диапазон
- `*` – любое число повторений (в т.ч. 0)

```
In [229... eng_letters = re.compile('[a-zA-Z]*')

print(eng_letters.match(''))
print(eng_letters.match('qwerty'))
print(eng_letters.match('йцукен'))
print(eng_letters.match('QWERTY'))
print(eng_letters.match('qweкен'))

<re.Match object; span=(0, 0), match=''>
<re.Match object; span=(0, 6), match='qwerty'>
<re.Match object; span=(0, 0), match=''>
<re.Match object; span=(0, 6), match='QWERTY'>
<re.Match object; span=(0, 3), match='qwe'>
```

Регулярные выражения

- Букву Ё прописываем отдельно, она вне диапазона
- Как в обычных строках, спецсимволы вносим как обычные с помощью `\`
- `+` – любое число повторений, но не менее одного

```
In [230... eng_letters = re.compile('[a-zA-Za-яА-ЯёЁ\\- .@]+')

print(eng_letters.match(''))
print(eng_letters.match('йцукен'))
print(eng_letters.match('Ёжик'))
print(eng_letters.match('Фрекен-Бок'))
print(eng_letters.match('mail@ya.ru'))
```

```
None
<re.Match object; span=(0, 6), match='йцукен'>
<re.Match object; span=(0, 4), match='Ёжик'>
<re.Match object; span=(0, 10), match='Фрекен-Бок'>
<re.Match object; span=(0, 10), match='mail@ya.ru'>
```

Регулярные выражения

- `^` – обозначает необходимость начала строки при совпадении
- `$` – обозначает необходимость конца строки при совпадении
- `{}` – после квадратных скобок позволяет ограничить количество вхождений символов

```
In [231... phone_pattern = re.compile('^(\+[0-9]{1} ?\((? [0-9]{3}\)) ? [0-9]{7})+$')

print(phone_pattern.match('89167698275'))
print(phone_pattern.match('+7 (916) 7698275'))
print(phone_pattern.match('+7(916)7698275'))
```

```
None
<re.Match object; span=(0, 16), match='+7 (916) 7698275'>
<re.Match object; span=(0, 14), match='+7(916)7698275'>
```

```
In [232... email_pattern = re.compile(r'^[A-Za-z0-9\.\_\-]+@[A-Za-z0-9\.\_\-]+\.[a-zA-Z]*')

print(email_pattern.match('one-two@yandex.ru'))
print(email_pattern.match('one&two@yandex.ru'))
print(email_pattern.match('one-two@yandex'))
```

```
<re.Match object; span=(0, 17), match='one-two@yandex.ru'>
```

```
None
```

```
None
```

Регулярные выражения

```
In [233... email_pattern = re.compile(r'[A-Za-z0-9\.\_\-]+@[A-Za-z0-9\.\_\-]+\.[a-zA-Z]*')

print(email_pattern.match('one-two@yandex.ru one-two@yandex.ru'))
email_pattern.findall('one-two@yandex.ru one-two@yandex.ru')
```

```
<re.Match object; span=(0, 17), match='one-two@yandex.ru'>
```

```
Out[233]: ['one-two@yandex.ru', 'one-two@yandex.ru']
```

Вместо конкретных символов можно использовать общие обозначения:

- `\s` – один пробел
- `\S` – один не пробел
- `\w` – один alpha-numeric символ

```
In [234... re.split('\w', '&a%6!7#b*')
```

Out[234]: ['&', '%', '!', '#', '*']

Идентификатор объекта

- у каждого объекта в Python есть целочисленный идентификатор
- на время жизни объекта он является уникальным
- получить идентификатор объекта можно с помощью функции `id`

```
In [235... a = 10
b = 11
c = b
a += 1
print(id(a), id(b))
print(a == b)
print(a is b)
```

```
140616734165616 140616734165616
True
True
```

- идентификатор зависит от реализации, это может быть адрес в памяти
- у одинаковых константных объектов этот идентификатор всегда одинаковый

```
In [236... a, b = 10, 10
c, d = 12, 12
print(id(a), id(b), '\n')
print(id(c), id(d))
```

```
140616734165584 140616734165584

140616734165648 140616734165648
```

Идентификатор константного объекта

- таким образом, одинаковые константы хранятся в единственном экземпляре
- Python делает так для экономии памяти
- типы в Python тоже являются объектами

```
In [237... print(id(type(12)), id(type(5)), id(int), '\n')
print(id(None), id(type(None)))
a = [5]
b = [5]
print(id(a), id(b))
```

```
94233239152608 94233239152608 94233239152608

94233239141632 94233239141216
140616626195200 140616626200704
```

- проверка равенства идентификаторов делается с помощью оператора `is`

```
In [238... a = None
print(a is None)
print(a == None)
```

True

True

Идентификатор изменяемого объекта

- два одинаковых списка не идентичны:

```
In [239... a = [1, 2, 3]
b = [1, 2, 3]
a is b
```

Out[239]: False

- две ссылки на один список идентичны:

```
In [240... c = a
a is c
```

Out[240]: True

- и все между собой, очевидно, равны:

```
In [241... a == b == c
```

Out[241]: True

Почему на None надо проверять с помощью is

```
In [242... # https://stackoverflow.com/questions/3257919/what-is-the-difference-between

class Foo:
    def __eq__(self, other): # magic method: == operator
        return True

foo=Foo()

print(foo == None)

print(foo is None)
```

True

False

- кроме того, проверка на идентичность в общем случае быстрее, чем на эквивалентность

Стадии жизни объекта

- Создание объекта приводит к выделению фрагмента памяти
- У каждого объекта есть счётчик ссылок
- Каждое новая ссылающаяся переменная увеличивает этот счётчик на единицу
- Ссылку можно удалить с помощью оператора `del`
- Когда счётчик ссылок на объект становится равным 0, объект удаляется
- Удаление и создание объектов может не приводит к изменениям памяти в некоторых случаях из-за кэширования (например 0 или None)
- Но в общем случае память выделяется и освобождается

Сборщик мусора

- Освобождение памяти производит сборщик мусора (Garbage Collector)
- Памяти освобождается не моментально, а в некоторый недетерминированный момент
- Одной из сложностей поиска объектов для удаления являются т.н. циклические ссылки

```
In [243... a = [None]
a[0] = a
del a
```

- Сборщик мусора умеет работать с такими случаями

Полезные функции

```
In [244... import gc
gc.collect() # call collection immediately
```

```
Out[244]: 2805
```

```
In [245... import sys
```

```
a = 1
print(sys.getrefcount(a)) # cached popular value
print(sys.getrefcount('some str')) # 1 after creation, 1 temporary, 1 - ???
```

9084
3

Слабые ссылки

- Иногда возникает необходимость вести учёт объектов только когда они используются где-нибудь ещё
- Но само слежение уже создаёт ссылку, из-за которой объект будет оставаться в памяти
- Слабые ссылки дают возможность вести учёт объекта без создания ссылок на них
- Для этого используется модуль weakref
- Классическое приложение – кэширование объектов, пересоздание которых является затратным
- Слабые ссылки нельзя создавать на объекты базовых типов

In [246... [# http://old.pymsk.ru/posts/2015/Sep/19/tainstvo-standartnoi-biblioteki-slab](http://old.pymsk.ru/posts/2015/Sep/19/tainstvo-standartnoi-biblioteki-slab)

```
import weakref, gc

class A:
    def __init__(self, value):
        self.value = value
    def __repr__(self): # magic method: information for print
        return str(self.value)

a = A(10) # create object and strong reference
d = weakref.WeakValueDictionary() # dict with weak references

d['k'] = a
d['k']
```

Out[246]: 10

In [247...

```
del a
gc.collect()
d['k']
```

Out[247]: 10

- при запуске из терминала получается вывод "KeyError: 'k'"

- видимо, в ноутбуке создаются дополнительные неявные ссылки

Проверка стиля кода

- Стандартным набором правил codestyle для Python является PEP8
- Есть несколько инструментов для проверки кода на соответствие стиля: `pep8`, `pyflakes`, `flake8`, `Pylint`
- В рамках курса в качестве линтера используется `flake8`
- Установка модуля стандартная через `pip`, используется он как утилита с управляющими флагами:

[illegible]

```
In [252... !flake8 . --extend-exclude=tmp --show-source --statistics --count --max-line
shutil.rmtree('tmp')
```

[illegible]

Спасибо за внимание!